

Airframe: 5-minute pitch

Seven slides, about 700 spoken words. Every number here was re-checked against the captures or re-run (see `CROSSCHECK_REPORT.md`).

Before you present:

- Check the report's second line says identifiers are masked. Masking is the default; `--no-mask` would say "do not share".
- Have the dashboard open on the Incident tab.
- Re-time `airframe_analyze.py` on the demo laptop, because the speed numbers were measured on a cloud VM.

Slide plan

#	Time	Headline on the slide	Visual	Key facts on the slide
1	0:00–0:30	The Wi-Fi didn't break. The login server did.	Incident tape, alert line at 14:25:28	First failed login 14:25:23 → alert 14:25:28
2	0:30–1:00	Failures hide in sequences, not frames	One client's login loop (frame list)	Header-only; overlapping sensors; Wi-Fi blamed 64%, root cause 40%
3	1:00–2:10	Two problems, two teams, under a minute each	Findings list	63/63 clients, 26 APs; 18 devices kicked from 14:30:53; retry spike = traffic mix; IoT network healthy
4	2:10–2:50	Read, rebuild, judge, route	Four-step diagram	Masked at parse; two independent engines agree; 128 checks pass
5	2:50–3:55	Built for fleet scale	Edge → partitioned core → roll-up	1.1 M frames in 4.5 s on one core; edge data 1,800x smaller; same answer on 1 or 32 partitions
6	3:55–4:40	The product is time	Savings tab	~\$70k per site per year on user time; \$2.3 M/h idle line is the upside
7	4:40–5:00	It tells you when it isn't the Wi-Fi	Logo + one line	—

Script

Slide 1: hook (0:00)

At 14:25:23, the first device on this factory's enterprise network failed to log in. Replaying the sensor captures, Airframe raised a critical alert five seconds later. It named the cause and the team that should fix it. And the cause wasn't the Wi-Fi.

Slide 2: the problem (0:30)

Tesla's sensors already record the air, but only headers, no payloads. A failure doesn't show up as one bad frame. It shows up as a sequence that stops halfway. Sensors overlap, so every problem appears many times. And Wi-Fi gets blamed for everything: in one survey it was reported as the cause 64% of the time, but was the root cause only 40%. Teams lose hours on the wrong layer.

Slide 3: what we found (1:00) *(point at the tape)*

Eight sensors, thirty minutes, 1.1 million frames.

First: 63 out of 63 devices on the enterprise network never logged in, on 26 access points and every channel. The access points only ever ask "who are you?" and never start the actual login. Every client we could hear answered. So the authentication server isn't replying. That alert goes to the identity team.

Second: from 14:30:53, access points started kicking 18 devices every 11 to 30 seconds, all from one vendor. That alert goes to the controller team, 58 seconds after it began.

Third: at 14:27 the retry rate doubled. It looks like a radio problem, but it isn't: it's clients rescanning after failed logins. The RF team can stand down.

And the IoT network kept working: every successful join was an IoT device.

Slide 4: how it works (2:10)

Four steps.

1. **Read and mask:** we parse headers only, and pseudonymise every MAC and network name as it's read.
2. **Rebuild every join:** for each client, across all sensors, we replay authentication, association, login and key exchange, and record exactly where it stopped.
3. **Judge:** a roam isn't a failure, one client seen by three sensors is one finding, and shared failures become one incident.
4. **Route:** each alert says what we saw, what we can't prove, and who owns it.

We built two engines independently, one on Wireshark and one from scratch. They agree on every headline number and pass 128 checks, including eleven brand-new cases neither had seen before.

Slide 5: scale (2:50) *(slow down here)*

97% of the air is routine: beacons and probe responses. Only about 1% of frames need correlating. So the sensors do the work.

- **At the edge:** each sensor turns its capture into a compact event file. For our 8 sensors, 286 megabytes became 0.16 megabytes, and the answer came out identical.
- **In the core:** events are partitioned by client, so any number of workers can share the load. We get the same result on one partition or thirty-two.
- **Speed:** one core analyses all 1.1 million frames in about four and a half seconds.
- **At fleet scale:** extrapolating from this site's traffic, ten thousand sensors would send the core about a tenth of a megabyte per second.
- **Roll-up:** one server failing across three plants becomes one incident, not thousands of alarms.

Slide 6: value (3:55) *(Savings tab)*

The product is time: detection in seconds instead of the typical half hour. At conservative assumptions, counting user time alone, that's about \$70,000 per site per year. The real lever is production. An idle automotive line costs \$2.3 million an hour, and every 1% of incident time that touches a line adds about \$340,000 per site per year. No new hardware, no payload access. It even found free fixes: two overloaded channels and three sensors that almost never hear clients.

Slide 7: close (4:40)

Airframe doesn't just tell you the Wi-Fi is bad. It tells you when it isn't the Wi-Fi, and who to call. Thank you.

Numbers you can say, and where they come from

Say	Source
Alert 5 s after the first failed login (14:25:23 → 14:25:28)	Replay of the captures, fast engine
63/63 clients, 26 APs, 8 channels, no EAP-Success or EAP-Failure	Raw frames and both engines
18 devices, one vendor, from 14:30:53, every 11–30 s; alert after 58 s	Raw frames and both engines
Retry 4.5% → 9.5% while probe-response retries stay about 31%	Per-minute metrics (full minutes); the dashboard shows the same
1,118,853 frames in 4.4 s on one core (≈250k frames/s)	Measured on a cloud VM; re-time on the demo laptop
Edge files 0.16 MB from 286 MB (1,800x), identical results	Measured
10,000 sensors ≈ 0.1 MB/s into the core	Extrapolated from this site's traffic; incident storms add more
128 checks: 56 + 48 + 24	Test logs in <code>results/</code> and <code>sample_cases/results.txt</code>
~\$70k per site per year; +\$340k per 1% production exposure	Savings model; assumptions labelled in the dashboard

Q&A crib

- **"Is the 5 seconds live?"** No, it's a replay of the captures. The logic is a per-client state machine, so it streams naturally.
- **"How do you know it's the server and not bad passwords?"** Rejected credentials produce EAP-Failure frames, and there are none. The APs only repeat Identity requests until they time out, and every audible client answered.

- **"Why not alert on retry rate?"** It doubled at 14:27 with no radio change. Probe responses are always retried about 31% of the time, so a change in traffic mix moves the average.
- **"Two sensors on the same channel? Clock drift?"** Same frame within 1 s counts once. That's tested with a 0.3 s skew.
- **"What if a sensor stops recording?"** It's reported as a sensor gap, and no AP is blamed for it. That's tested.
- **"Spoofed disconnects?"** Floods are called floods. We don't yet verify sender sequence numbers, and the alert says so.
- **"Privacy?"** MACs keep only their vendor prefix, the rest is a keyed hash, SSIDs are hashed, and login identities are never read. The tests search every output for the raw values.
- **"Business impact?"** Modelled, not measured: the captures contain no application data. That's why the production figure starts at zero.